

Complete Study Guide

The Logic Advocate Tutor

Theory, Code, Math, and Practice Questions

Adel Ashraf Mohamed Kamel Emam — GUC, SS26

A comprehensive study document covering the theoretical foundations, the engine math, the MDP formalism, the code architecture, the evaluation methodology, and a self-test bank of likely supervisor questions.

Designed for repeated reading and viva preparation.

Table of Contents

PART 1. How to Use This Guide	3
PART 2. Theoretical Foundations	4
2.1 Dung (1995) Abstract Argumentation Framework	4
2.2 Besnard & Hunter (2001) h-categorizer	5
2.3 Amgoud & Cayrol (2002) Preference-based AF	6
2.4 Bench-Capon (2003) Value-Based AF	7
2.5 Cayrol & Lagasquie-Schiex (2005) Bipolar AF	8
2.6 Gradual semantics (Cayrol 2005; Amgoud 2017)	9
2.7 Stamper et al. (2011) Hint Factory	10
2.8 Ku et al. (2025) Implicit Premises in LLM Debate	11
PART 3. System Architecture	12
PART 4. The Bipolar Gradual Engine	14
4.1 Formal definition	14
4.2 The acceptability score formula	15
4.3 Worked examples	16
4.4 Code walkthrough	17
PART 5. The Hint MDP	19
5.1 The formal tuple	19
5.2 Why each component is what it is	20
5.3 Code walkthrough	21
PART 6. The AI Layer	23
6.1 The grounding mechanism	23
6.2 Rules A-D in the prompt	24
6.3 The pipe parser	25
6.4 The implicit premise detector	26
PART 7. Evaluation	27
7.1 The 60-scenario audit	27
7.2 The ablation study	28
7.3 Unit tests	29
7.4 Discrepancy analysis	30
PART 8. Self-Test Question Bank	31

Part 1 — How to Use This Guide

Goal

This document is your single-source study aid for the project. Read it cover-to-cover once, then return to specific parts before the viva. Each major section ends with a 'questions to test yourself' subsection.

Three reading modes

Cover-to-cover (3 hours)

Read everything once. You will not memorise the math on first pass; that's normal. The aim is to get the shape of the system in your head.

Topic dive (30-45 minutes per topic)

Pick one part. Read the theory, then the code, then the practice questions. Try to answer the questions without looking at the answer first.

Lightning review (10 minutes)

Skim the Vocabulary boxes (the gold callouts) and the Part 8 question bank. Useful the morning of the viva.

Symbols used in this guide

- **Italic grey box** = key claim you should be able to deliver verbatim.
- **Gold callout box** = vocabulary or formal definition. Memorise these.
- **Courier text** = actual code or formal notation.
- **Q. / A.** = practice question with model answer.

A word on study order

If you have limited time, work through this order:

- Part 4 (the engine) — this is the heart. If you understand the formula, half the viva is done.
- Part 5 (the MDP) — supervisor specifically asked about this.
- Part 6.1 (the grounding mechanism) — the central pedagogical claim.
- Part 7 (evaluation) — have the numbers memorised.
- Part 8 (Q&A) — rehearse the questions aloud.

Part 2 — Theoretical Foundations

Your system is not invented from scratch. It rests on eight published results from the argumentation literature and one from intelligent tutoring. This part explains each one in just enough depth to defend the citation. If asked 'why this paper?', you should be able to deliver a one-sentence answer.

2.1 Dung (1995) — Abstract Argumentation Frameworks

Full citation: Dung, P. M. (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2), 321-357.

What it says

Dung modelled a debate as a finite set of arguments and a binary attack relation between them. The entire framework is just two objects:

$F = \langle A, R_{att} \rangle$
A = finite set of arguments (abstract nodes)
 R_{att} = relation: $A \times A \rightarrow \{\text{attack}, \text{no-attack}\}$

The key insight is the move from 'arguments have internal structure' to 'arguments are abstract nodes whose only property is who attacks whom'. This abstraction is what makes the math tractable.

Dung's semantics (the IN/OUT verdict)

Dung also defines several ways to decide which arguments 'survive'. The one your project builds on is the **grounded extension**:

- An argument is IN if all its attackers are OUT.
- An argument is OUT if any of its attackers is IN.
- If neither condition is decided yet, it is UNDEC. Apply rules iteratively until stable.

This is the binary classical reading your project departs from.

Why your project cites it

Your engine's data model (nodes + attack relation) is a direct implementation of Dung's $F = \langle A, R \rangle$. When you turn off your engine's extras (supports, weights, value tags, gradual semantics), what remains is Dung's framework. So Dung is the floor your work stands on.

Likely viva question

Q. Why use Dung's framework as the basis instead of, say, Toulmin?

A. Two reasons. First, Dung's framework is the dominant formalism in computational argumentation: every subsequent extension (bipolar, value-based, gradual) builds on it, so I get a path to richer formalisms. Second, Dung treats arguments as abstract nodes, which means my engine can work with any natural-language argument without parsing its internal structure — that is exactly what I needed for a chat-style debate interface.

2.2 Besnard & Hunter (2001) — the h-categorizer

Full citation: Besnard, P., & Hunter, A. (2001). A logic-based theory of deductive arguments. *Artificial Intelligence*, 128(1-2), 203-235.

What it says

They introduced a numerical evaluation called the h-categorizer that assigns every argument a score in $[0, 1]$:

$$s(a) = 1 / (1 + \text{SUM over } (b, a) \text{ in } R_{\text{att}} \text{ of } s(b))$$

An isolated argument scores 1. An argument hit by a single fully-accepted attacker scores 1/2. The cascade approximates Dung's grounded extension after thresholding at 0.5.

Why this is important to your project

Your acceptability formula is a **generalisation** of the h-categorizer. Compare:

Besnard & Hunter: $s(a) = 1 / (1 + \alpha(a))$

Your engine: $s(a) = (1 + \sigma(a)) / (1 + \alpha(a))$
[capped at 1]

where α = sum of attacker contributions

σ = sum of supporter contributions

Set σ to zero, use uniform weights, and use one value tag, and your formula reduces to theirs. This is your **classical baseline**: it lets you claim your engine 'behaves like a published classical evaluator on classical inputs and only departs from it when extra features (bipolar, weighted, value-tagged) are activated.'

Likely viva question

Q. Why this specific functional form, the $(1+\sigma)/(1+\alpha)$ shape?

A. Three reasons. First, it generalises the published h-categorizer to the bipolar case — it has a paper as its limit. Second, it is bounded in $[0, 1]$ once I cap at 1, which keeps the colour gradient in the UI well-defined. Third, it is monotone in both its arguments: more support raises the score, more attack lowers it, which matches intuition.

2.3 Amgoud & Cayrol (2002) — Preference-Based AF

Full citation: Amgoud, L., & Cayrol, C. (2002). A reasoning model based on the production of acceptable arguments. *Annals of Mathematics and Artificial Intelligence*, 34, 197-215.

What it says

Pure Dung treats every argument as equally important. Amgoud and Cayrol added **preferences**: each argument has a strength (a number), and an attacker only succeeds in defeating its target if its strength is at least as great:

$a \text{ defeats } b \iff \text{attacks}(a, b) \text{ AND } \text{strength}(a) \geq \text{strength}(b)$

Why your project cites it

Your integer weight $w(a)$ in $[1, 25]$ implements this. Heavier arguments resist defeat more strongly. The original Amgoud-Cayrol rule was binary (if your weight is below the target's, the attack is silently dropped), and your prototype *class Argument.txt* used exactly that strict rule.

Why you softened the rule in the deployed engine

The strict rule was too brittle in practice. The LLM would sometimes fire a weak but rhetorically clever attack against a heavy claim, and the prototype would refuse to register the move — the disagreement disappeared. So in the deployed engine I softened the rule into a continuous ratio: a weaker attacker still erodes the target, but cannot drive its score below $1/(1 + \alpha)$. Pedagogically this is preferable because the learner sees the attack land and can decide how worried to be.

Likely viva question

Q. Why integer weights 1-25 specifically? Why not floats?

A. Two practical reasons. First, the AI scores the persuasiveness of every argument on a 1-25 scale; treating the LLM's score as the weight gives me a single consistent number. Second, integers make the UI easier to read — a weight of 17 is more comprehensible than 0.687. The choice has no theoretical content; the engine would work identically with any positive numeric weight.

2.4 Bench-Capon (2003) — Value-Based AF

Full citation: Bench-Capon, T. J. M. (2003). Persuasion in practical argument using value-based argumentation frameworks. *Journal of Logic and Computation*, 13(3), 429-448.

What it says

Bench-Capon noticed that arguments often appeal to different **values** — Fact, Fairness, Liberty, Security, etc. — and different **audiences** care about different values. So whether an attack succeeds depends not just on logic but on whose values you privilege. He introduced an audience preference order over values, and attacks only succeed if the attacker's value is at least as preferred as the target's.

Why your project cites it

Your four value tags (Fact, Logic, Ethics, Emotion) are a simplified Bench-Capon value set. Your multipliers (1.2 / 1.1 / 1.0 / 0.8) encode an audience preference: a Fact-leaning rational audience finds Fact-tagged arguments more persuasive than Emotion-tagged ones. The ratio $\mu(v(b)) / \mu(v(a))$ in your formula scales the attack/support strength accordingly.

Why these specific multipliers?

Hand-tuned by inspection. I picked them so that a Fact attack on an Emotion target is roughly 50% more damaging than the reverse ($1.2/0.8 = 1.5$), which lines up with the asymmetry Bench-Capon describes in his worked examples. Future work in Chapter 6 includes learning these multipliers from tournament data per classroom.

Likely viva question

Q. So you encode a Fact-leaning audience by default. Doesn't that bias the system?

A. Yes, deliberately. Most academic debate audiences are fact-leaning, so this default matches the most common use case. But the multipliers are not hard-coded — they are a single Python dict. A teacher could rebalance them per classroom: for a moral-reasoning class, set Ethics highest; for a creative-writing class, set Emotion highest. This is exactly the audience-relative reading Bench-Capon intended.

2.5 Cayrol & Lagasquie-Schiex (2005) — Bipolar AF

Full citation: Cayrol, C., & Lagasquie-Schiex, M.-C. (2005). On the acceptability of arguments in bipolar argumentation frameworks. *Symbolic and Quantitative Approaches to Reasoning with Uncertainty*, Lecture Notes in Computer Science 3571, 378-389. Springer.

What it says

Pure Dung is adversarial only: arguments either attack or do nothing. But real debate also has **support**: arguments build on each other, reinforce the same claim from different angles. Cayrol and Lagasquie-Schiex added a parallel support relation:

$F = \langle A, R_{att}, R_{sup} \rangle$
 $R_{sup} = \text{relation: } A \times A \rightarrow \{\text{support, no-support}\}$

A support edge from b to a means b reinforces a . They also defined how supports and attacks combine to produce a 'bipolar' verdict.

Why your project cites it

Your engine has supports as a first-class relation alongside attacks. The $\sigma(a)$ sum in your formula is exactly the bipolar reinforcement contribution. Crucially, your ablation study shows that disabling supports flips **26.7%** of verdicts — so bipolarity is not decorative.

2.6 Gradual Semantics — Cayrol (2005) & Amgoud et al. (2017)

Full citations:

Cayrol, C., & Lagasque-Schiex, M.-C. (2005). Graduality in argumentation. *Journal of Artificial Intelligence Research*, 23, 245-297.

Amgoud, L., Ben-Naim, J., Doder, D., & Vesic, S. (2017). Acceptability semantics for weighted argumentation frameworks. *IJCAI 2017*.

What it says

Classical Dung returns binary verdicts: IN or OUT. Gradual semantics replaces the binary verdict with a **continuous score** in $[0, 1]$ computed by a fixed-point iteration. The point: pedagogically, telling a student 'your argument is OUT' is useless — they don't know if it was close or hopeless. A score of 0.42 (just below 0.5) is much more informative.

Why your project cites it

Your engine is a member of the gradual-semantics family. Specifically:

- Your scores are in $[0, 1]$.
- You compute them by 10 rounds of synchronous relaxation (fixed-point).
- You only threshold to IN/OUT at the end for backwards UI compatibility.
- The ablation study shows replacing your gradual semantics with classical Dung-grounded changes **41.7%** of verdicts. So the choice matters.

Likely viva question

Q. Convergence: how do you know your relaxation loop terminates?

A. Three pieces of evidence. First, empirically on the 60 scenarios, the score vector stops changing within 2-3 iterations, far short of the 10-iteration budget. Second, the formula is monotone-bounded: every iteration is bounded in $[0, 1]$ by the cap, so the sequence cannot diverge. Third, a formal proof via Banach's fixed-point theorem (the map is a contraction under modest assumptions) is in the future-work section of Chapter 6 — I gestured at it but did not derive it formally.

2.7 Stamper et al. (2011) — the Hint Factory

Full citation: Stamper, J., Eagle, M., Barnes, T., & Croy, M. (2011). Experimental Evaluation of Automatic Hint Generation for a Logic Tutor. *Proceedings of AIED 2011*, 345-352.

What it says

Stamper et al. built a logic tutor that learns hint policies from historical student data and asks 'when a student is in state X, which hint helped most in the past?' They model it as a Markov Decision Process and derive the policy from the transition data. Two design principles fall out:

- **On-demand** — hints appear only when the student asks. Surprise hints break flow.
- **Non-evaluative** — hints suggest direction without saying 'you are wrong'. They don't punish help-seeking.

Their empirical finding: students with access to the hint factory completed more problems and were less likely to abandon the tutor.

Why your project cites it

Your hint system is a Markov Decision Process directly inspired by their design principles. You operationalise on-demand (the hint button is opt-in) and non-evaluative (the hint suggests a strategy, not a verdict). The fatigue multiplier in your transition probability matches their non-evaluative principle — it discourages spamming without penalising legitimate help-seeking.

How your MDP differs from theirs

Theirs learns the transition model from historical student trajectories. Yours uses an expert-informed initial transition model because you don't have logged trajectories yet. The empirical re-estimation of P from logged data is a stub in your codebase (see `update_transition_model_from_logs` in `hint_mdp.py`) and is explicit future work.

2.8 Ku et al. (2025) — Implicit Premises in Multi-Agent LLM Debate

Full citation: Ku, H. B., Shin, J., Lee, H. J., Na, S., & Jeon, I. (2025). Multi-Agent LLM Debate Unveils the Premise Left Unsaid. *Proceedings of the 12th Argument Mining Workshop*, 58-73, ACL.

What it says

Ku et al. studied what happens when LLMs are assigned debate stances. Two findings important to your work:

- **Rhetorical rigidity:** once assigned a stance, LLMs defend flawed reasoning with confidence rather than admit a defeated premise.
- **Implicit premise blindness:** LLMs (and humans) tend to skip the unstated assumption their reasoning rests on.

Why your project cites it

Two layers of contribution.

- **Diagnostic alignment:** Ku's rhetorical-rigidity finding is the motivation for your entire grounding mechanism. You don't try to fix the LLM's rigidity — you make the LLM unable to lie about the verdict by depriving it of the privilege of computing it.
- **Operationalising the implicit-premise finding:** Ku diagnosed that implicit premises get missed; your Check Assumptions button is a UI feature that surfaces them. The user can accept the suggested premise and the system adds it as an explicit supporting node. This makes implicit reasoning visible in the engine's graph, which is more than what Ku himself does — he diagnoses, you treat.

This is your **novel contribution** on top of the literature, worth mentioning in the viva.

Likely viva question

Q. If Ku et al. diagnose the problem, why is your fix new?

A. Ku et al. measure how often LLMs miss implicit premises in adversarial debate. They do not propose a treatment. My system flips the diagnosis into a UI feature: before submission, the learner can ask the LLM to identify the unstated assumption their argument rests on, and if they accept it, the premise becomes an explicit supporting node in the engine. So the implicit reasoning gets surfaced into the formal graph rather than buried in the prose. I'm not aware of any other deployed argumentation tutor that does this.

Part 3 — System Architecture

The system has four layers, with a clear Symbolic/Neural separation. This is the diagram your supervisor explicitly asked for.

The four layers

Presentation Layer (Hybrid — gray)

Files: `views/local_debate.py`, `views/ai_vs_ai.py`, `views/engine_inspector.py`, `views/_styles.py`.

Renders chat bubbles, the momentum bar, the Graphviz logic map, the Inspector panel, the composer. Reactive: re-renders from session state on every interaction.

Dialogue Layer (Hybrid — gray)

File: `app.py` + parts of the views.

Enforces turn order, validates moves, runs the optional 60-second blitz timer, routes between the three game modes.

Logic Layer (Symbolic — gold)

Files: `logic_engine.py` and `hint_mdp.py`.

Pure deterministic Python. Implements the bipolar gradual evaluator and the Markov Decision Process for hint strategy. Knows nothing about the user, the LLM, or the network. Unit-testable in isolation.

AI Layer (Neural — red)

File: `ai_agent.py`.

Wraps Groq calls to LLaMA-3 (text and vision) and Whisper (voice). Outputs text and integer scores. Receives prompts that include the transcript but NOT the engine's verdict. This is the boundary that enforces the pedagogical guarantee.

VOCABULARY: pedagogical guarantee. The claim that 'the LLM cannot lie about who is winning' because the engine's verdict is computed by deterministic Python after every move, and the UI only reads the engine. The LLM never has access to the verdict, so it cannot align its text to it.

Practice questions

Q. Where is the boundary between Symbolic and Neural enforced?

A. Two places. First, `ai_agent.py` never reads the engine's scores or statuses — only the transcript is passed in. Second, the views always read scores from `engine.scores` and `engine.statuses` for every UI widget — never from any LLM-supplied field. The pipe-separated protocol carries LLM outputs into the engine but never the other way.

Q. Why is the Dialogue Layer separate from the Presentation Layer?

A. Because the same dialogue rules (turn order, move validation, end-of-debate) apply across all three game modes, but the presentation differs. Keeping Dialogue separate lets me share the engine update logic between the three views without duplicating it.

Part 4 — The Bipolar Gradual Engine

4.1 Formal definition

A debate is represented as a tuple:

$F = \langle A, R_{att}, R_{sup}, w, v \rangle$

Where:

- A = finite set of arguments (nodes).
- $R_{att} \subseteq A \times A$ = attack relation.
- $R_{sup} \subseteq A \times A$ = support relation.
- $w : A \rightarrow \{1, 2, \dots, 25\}$ = integer weight per argument.
- $v : A \rightarrow \{\text{Fact, Logic, Ethics, Emotion}\}$ = value tag per argument.

Each value tag has a hand-tuned multiplier:

```
mu(Fact)      = 1.2
mu(Logic)     = 1.1
mu(Ethics)    = 1.0
mu(Emotion)   = 0.8
```

4.2 The acceptability score formula

Every argument a gets a score $s(a)$ in $[0, 1]$ computed as the fixed point of:

```
alpha(a) = SUM over (b, a) in R_att of s(b) * w(b) * mu(v(b)) / mu(v(a))
sigma(a) = SUM over (b, a) in R_sup of s(b) * w(b) * mu(v(b)) / mu(v(a))
```

```
s_(t+1)(a) = min( 1, (1 + sigma(a) / w(a)) / (1 + alpha(a) / w(a)) )
```

After convergence, threshold:

```
status(a) = IN if s(a) >= 0.5
           OUT otherwise
```

VOCABULARY: synchronous relaxation. An iterative fixed-point algorithm where every node is updated using the scores from the previous iteration, not the in-progress values from the current iteration. This makes the result independent of the order in which nodes were added to the graph, which is important for reproducibility.

4.3 Worked examples

Example 1 — classical limit (no support, uniform weight, single tag)

Take F with one argument a_1 (Main Claim) and one attacker a_2 :

```
a_1: w=1, tag=Ethics    (mu = 1.0)
a_2: w=1, tag=Ethics    (mu = 1.0)
R_att = {(a_2, a_1)}
R_sup = {}
```

Initialise $s(a_1) = s(a_2) = 1$. Iterate:

```
alpha(a_1) = s(a_2) * w(a_2) * mu(Ethics)/mu(Ethics) = 1 * 1 * 1 = 1
sigma(a_1) = 0
s(a_1) = min(1, (1 + 0)/(1 + 1)) = 1/2 = 0.5
```

a_2 has no attackers: $s(a_2) = 1$, stays 1.

After convergence: $s(a_1) = 0.5$, status IN (threshold is ≥ 0.5).

This matches the h-categorizer of Besnard & Hunter exactly. So your engine produces the published classical result on classical inputs.

Example 2 — value-tag asymmetry

Same graph but change the tags:

```
a_1: w=1, tag=Fact      (mu = 1.2)
a_2: w=1, tag=Emotion   (mu = 0.8)
```

```
alpha(a_1) = 1 * 1 * 0.8/1.2 = 0.667
s(a_1) = (1 + 0)/(1 + 0.667) = 1/1.667 = 0.60 -> IN
```

Swap the tags:

```
a_1: w=1, tag=Emotion   (mu = 0.8)
a_2: w=1, tag=Fact      (mu = 1.2)
```

```
alpha(a_1) = 1 * 1 * 1.2/0.8 = 1.5
s(a_1) = 1/(1 + 1.5) = 0.40 -> OUT
```

So the same attacker/target pair flips the verdict depending on which side is factual and which is emotional. This is Bench-Capon's audience model in action.

Example 3 — bipolar support saves a defeated claim

Take F with one claim under attack, plus a supporter:

```
a_1: w=10, tag=Logic     (mu = 1.1)
a_2: w=10, tag=Logic     (Atk a_1)
a_3: w=10, tag=Fact      (Sup a_1)
```

First iteration:

```
alpha(a_1) = 1 * 10 * 1.0 = 10
sigma(a_1) = 1 * 10 * 1.2/1.1 = 10.91
s(a_1) = (1 + 10.91/10)/(1 + 10/10) = 2.091/2 = 1.05 -> capped at 1.0 -> IN
```

Without a_3 , the same a_1 would score $(1+0)/(1+1) = 0.5$, exactly at threshold. With the Fact-tagged supporter, it stays clearly IN. This is exactly the kind of case where the ablation study (Section 7.2) flips the verdict when bipolarity is disabled.

4.4 Code walkthrough

The complete logic engine is in `logic_engine.py`. Here are the essential pieces.

The data class

```
class AcademicLogicEngine:
    def __init__(self):
        self.nodes = {} # mid -> {text, weight, value_tag}
        self.attacks = [] # list of (attacker_id, target_id)
        self.supports = [] # list of (supporter_id, target_id)
        self.statuses = {} # mid -> "IN" | "OUT"
        self.scores = {} # mid -> float in [0, 1]
```

The semantics loop

This is the fixed-point relaxation. Note that it uses a separate `new_scores` dict to make the update synchronous — reads from `self.scores`, writes to `new_scores`, swaps at the end of each iteration.

```
def evaluate_semantics(self):
    self.scores = {mid: 1.0 for mid in self.nodes}
    for _ in range(10):
        new_scores = {}
        for mid in self.nodes:
```

```

t_val = self.nodes[mid].get("value_tag", "Logic")
target_weight = max(1, self.nodes[mid]["weight"])

attack_sum = 0.0
for atk, tgt in self.attacks:
    if tgt == mid and atk in self.nodes:
        a_val = self.nodes[atk].get("value_tag", "Logic")
        mult = VALUE_WEIGHTS[a_val] / VALUE_WEIGHTS[t_val]
        attack_sum += self.scores[atk] * \
            self.nodes[atk]["weight"] * mult

support_sum = 0.0
for sup, tgt in self.supports:
    if tgt == mid and sup in self.nodes:
        s_val = self.nodes[sup].get("value_tag", "Logic")
        mult = VALUE_WEIGHTS[s_val] / VALUE_WEIGHTS[t_val]
        support_sum += self.scores[sup] * \
            self.nodes[sup]["weight"] * mult

score = (1.0 + support_sum / target_weight) / \
    (1.0 + attack_sum / target_weight)
new_scores[mid] = min(1.0, score)
# Early termination if converged
max_delta = max(abs(new_scores[m] - self.scores[m]) for m in self.nodes)
self.scores = new_scores
if max_delta < 1e-6:
    break

for mid in self.nodes:
    self.statuses[mid] = "IN" if self.scores[mid] >= 0.5 else "OUT"

```

Practice questions

Q. Why synchronous and not asynchronous relaxation?

A. Synchronous makes the result independent of node insertion order. With asynchronous updates, the order in which you visit nodes within an iteration would affect the converged state, which is bad for reproducibility and bad for debugging. Asynchronous would converge faster but with this trade-off.

Q. Why a fixed 10-iteration budget? What if it doesn't converge?

A. I include an early-termination check — if max delta between iterations is below 1e-6, I break early. Empirically on the 60 scenarios it converges in 2-3 iterations. The 10-iteration ceiling is a safety net; I have not encountered an input that needs more. A formal convergence proof is future work.

Q. What if all attackers and supporters have score 0?

A. Both alpha and sigma become 0, so the score is $(1+0)/(1+0) = 1$. An unattacked, unsupported argument is fully accepted. This is consistent with Dung's convention.

Part 5 — The Hint MDP

5.1 The formal tuple

The hint system is a standard textbook Markov Decision Process $\langle S, A, P, R, \gamma \rangle$.

VOCABULARY: Markov Decision Process (MDP). The standard formalism for sequential decision making under uncertainty. Defined by states, actions, transition probabilities ($P(s' | s, a)$), reward function ($R(s, a, s')$), and a discount factor (γ in $[0, 1]$) controlling how much future reward matters.

State space S

18 discrete states, the Cartesian product of three buckets:

- **own_status** in {WINNING, TIED, LOSING} — from the engine's IN/OUT verdict on the learner's main claim.
- **opponent_pressure** in {LOW, HIGH} — HIGH if the opponent's most recent argument scores at least 0.65.
- **hint_streak** in {ZERO, ONE, TWO_PLUS} — how many hints used since the learner's last actual move.

Total: $3 \times 2 \times 3 = 18$ states.

Action space A

Four hint strategies:

- **STRATEGIC_ATTACK** — attack the opponent's weakest premise
- **DEFENSIVE_REINFORCE** — strengthen the learner's own claim
- **VALUE_REFRAME** — suggest using a different value tag
- **STEP_BACK** — reconsider the main claim itself

Transition probability P

Expert-informed initial model encoding four intuitions: STRATEGIC_ATTACK helps most when LOSING, DEFENSIVE_REINFORCE when TIED, VALUE_REFRAME counters HIGH pressure, STEP_BACK is the rescue action. Plus a fatigue penalty for consecutive hints (Stamper-style non-evaluative principle).

Reward R

The link to logical validity.

```
R(s, a, s') = +1 if next own_status = WINNING
              0 if next own_status = TIED
              -1 if next own_status = LOSING
              minus 0.05 per hint (fatigue cost)
```

Crucially, own_status is computed by the engine's gradual semantics on the learner's main claim. So the reward is the engine's logical-validity verdict, with a small penalty for hint-spamming.

Discount γ

0.9, the standard textbook value.

5.2 Why each component is what it is

Why 18 states and not 100?

Three reasons. First, value iteration on 18 states is instantaneous — the policy is computed at module import time and cached. Second, 18 captures the minimum strategic context: am I winning, is the opponent strong, am I overusing hints. Third, fewer states means each state has more observations, so empirical re-estimation of P later will be statistically meaningful with realistic data.

Why these four actions and not five or three?

Each one maps cleanly to a dialectical move: attack (STRATEGIC_ATTACK), defend (DEFENSIVE_REINFORCE), reframe (VALUE_REFRAME), retract (STEP_BACK). A fifth action would either duplicate one of these or introduce an option my prompts cannot reliably distinguish. Three would lose the value-reframe option which is the one that uses the engine's value tags directly.

Why the reward is tied to engine verdict and not, say, hint usefulness?

Hint usefulness is what the LLM might rate, and exactly the kind of signal I want to avoid trusting. The engine's verdict is verifiable and independent of the LLM. If a hint leads to a learner move that the engine then judges WINNING, that hint was useful by the only criterion the system actually trusts.

5.3 Code walkthrough

The MDP lives entirely in hint_mdp.py, about 200 lines.

The state space construction

```
OWN_STATUS_LEVELS = ("WINNING", "TIED", "LOSING")
OPPONENT_PRESSURE = ("LOW", "HIGH")
HINT_STREAK_LEVELS = ("ZERO", "ONE", "TWO_PLUS")

STATES = [(s, p, h)
           for s in OWN_STATUS_LEVELS
           for p in OPPONENT_PRESSURE
           for h in HINT_STREAK_LEVELS]
# len(STATES) == 18
```

The transition probability

```
_LANDING_PROBABILITY = {
    "STRATEGIC_ATTACK": {"WINNING": 0.20, "TIED": 0.45, "LOSING": 0.55},
    "DEFENSIVE_REINFORCE": {"WINNING": 0.30, "TIED": 0.55, "LOSING": 0.25},
    "VALUE_REFRAME": {"WINNING": 0.25, "TIED": 0.40, "LOSING": 0.35},
    "STEP_BACK": {"WINNING": 0.10, "TIED": 0.30, "LOSING": 0.60},
}

def _transition_distribution(state, action):
    own, pressure, streak = state
    p = _LANDING_PROBABILITY[action][own]
    if pressure == "HIGH":
        p *= 1.1 if action == "VALUE_REFRAME" else 0.7
    if streak == "ONE":
        p *= 0.85
    elif streak == "TWO_PLUS":
        p *= 0.65
    p = max(0.05, min(0.85, p))
    # ... distribute remaining probability mass over next-states ...
```

The reward function

```
def reward(state, action, next_state):
    base = REWARD_BY_NEXT_STATUS[next_state[0]] # +1, 0, or -1
    return base + HINT_COST # HINT_COST = -0.05
```

The solver (value iteration)

```
def solve():
```

```

V = {s: 0.0 for s in STATES}
while True:
    new_V = {}
    for s in STATES:
        best = max(
            sum(prob * (reward(s, a, ns) + DISCOUNT * V[ns])
                for ns, prob in _transition_distribution(s, a).items())
            for a in ACTIONS
        )
        new_V[s] = best
    if max(abs(new_V[s] - V[s]) for s in STATES) < EPSILON:
        return V, derive_policy(V)
V = new_V

```

The policy comes back as a dict mapping state to optimal action. Cached at module import time. The hint button just does policy[state].

Practice questions

Q. How is your MDP different from Stamper's Hint Factory?

A. Three differences. First, Stamper's transition model is learned from logged student trajectories; mine is expert-informed because I don't have logged data yet, with a stub function for empirical re-estimation. Second, Stamper's reward is implicit in the historical trajectories; mine is explicitly tied to the engine's logical-validity verdict. Third, Stamper's action space is specific hint texts; my action space is hint STRATEGIES, and the LLM writes the prose. So my MDP is a layer above the LLM, not a replacement for it.

Q. What if a learner clicks the hint button rapidly five times in a row?

A. The hint_streak bucket transitions ZERO -> ONE -> TWO_PLUS and stays there. The fatigue multiplier in the transition probability shrinks every consecutive hint's landing probability (multiply by 0.85 for the second, 0.65 for the third and onward). So the MDP increasingly favours STEP_BACK or VALUE_REFRAKE — strategies meant to change the learner's situation rather than incremental advice. This implements Stamper's non-evaluative principle.

Part 6 — The AI Layer

6.1 The grounding mechanism

This is the most important part of the entire system to be able to defend.

The LLM is never asked who is winning. It writes prose, and it rates the persuasiveness of arguments on a 1-25 scale, but those ratings are inputs to the engine, not the engine's outputs. After every move, `evaluate_semantics` runs on the engine and recomputes the verdict deterministically. The UI reads the engine's verdict, not the LLM's claim.

So when a learner makes a knockout argument, the engine sets the AI's previous node to OUT, and the UI shows that OUT verdict, regardless of how confidently the AI's text was phrased. The LLM cannot 'lie' because it is not in a position to make the claim that the verdict-display would contradict.

How it is enforced in code

```
# In views/local_debate.py:
def _process_move(text, value_tag, target, action, active_side, ...):
    mid = f"Msg_{st.session_state.msg_counter}"
    engine.add_argument(mid, text, weight, value_tag)
    if action == "Attack":
        engine.add_direct_attack(mid, target)
    else:
        engine.add_support(mid, target)
    # The engine runs IMMEDIATELY after every move:
    engine.evaluate_semantics()
    # The UI then reads engine.scores and engine.statuses, never an LLM field.
```

6.2 Rules A-D in the prompt

The counter-argument prompt is wrapped in four explicit rules to mitigate common LLM failure modes.

Rule A — Accept personal experience as valid evidence

The LLM is told that 'I drink 1 litre a day' is testimony, not a research paper. It must not demand a citation for a self-report. This addresses the pedantic-tutor failure I observed where the AI would reject 'I work out three times a week' on the grounds of no peer-reviewed study.

Rule B — Score honestly

The LLM is explicitly told it has a self-favouring bias and must resist it. If the learner made a clear, reasonable point, the AI should rate them 15-22, not 8. If the AI's counter is weak, it should rate itself 5-10, not 19.

Rule C — Concede when there is no genuine counter

If the only way to attack the learner's argument is to be pedantic or split hairs, the AI must output the CONCEDE sentinel: '0|0|Logic|CONCEDE'. The engine then ends the debate.

Rule D — Stay focused

Ignore typos, casual phrasing, small grammar issues. Engage with the actual claim.

VOCABULARY: CONCEDE sentinel. The exact string '0|0|Logic|CONCEDE'. When the LLM emits this, the parser detects it and the dialogue layer ends the debate immediately. This is the AI's only way to honestly admit defeat. It is what makes the supervisor demo feel fair.

6.3 The pipe parser

The LLM is asked to return exactly one line in the format HumanScore|AIScore|ValueTag|CounterText. The parser is forgiving because LLMs sometimes wrap responses in preamble.

```
_PIPE_RE = re.compile(
    r"(\d{1,2})\s*\|\s*(\d{1,2})\s*\|\s*"
    r"(Logic|Fact|Ethics|Emotion)\s*\|\s*([^\n]+)",
    re.IGNORECASE,
)

def _parse_counter_response(text):
    matches = _PIPE_RE.findall(text)
    if matches:
        hw_s, aw_s, tag, txt = matches[-1] # Last match (in case of preamble)
        # ... clamp scores to [1, 25], capitalise tag, check for CONCEDE ...
        return {"human_weight": hw, "llm_weight": aw,
                "llm_val": tag, "llm_text": txt, "parsed": True}
    # Fallback: use the last line of the response (not the whole thing)
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    fallback = lines[-1] if lines else text.strip()
    return {"human_weight": 12, "llm_weight": 12, "llm_val": "Logic",
            "llm_text": fallback, "parsed": False}
```

If the LLM wraps the answer in five lines of explanation followed by the proper pipe line, the regex still finds the last pipe line and uses it. If the LLM fails to emit any pipe line at all, the fallback uses only the last line, so paragraphs of chain-of-thought do not leak into the chat bubble.

6.4 The implicit premise detector

The Check Assumptions button calls a separate function `detect_implicit_premise()` with a stricter prompt and lower temperature (0.4 vs 0.7). The expected response is three pipe-separated fields: PREMISE_TEXT | CONFIDENCE | JUSTIFICATION.

```
def detect_implicit_premise(user_text, recent_messages=None):
    prompt = _PREMISE_PROMPT.format(user_text=user_text, ...)
    response = client.chat.completions.create(
        model=TEXT_MODEL, messages=[{"role": "user", "content": prompt}],
        temperature=0.4, max_tokens=200,
    )
    raw = response.choices[0].message.content.strip()
    parts = raw.split("|", 2)
    if parts[0].upper().startswith("NONE") or float(parts[1]) < 0.10:
        return {"found": False, ...}
    return {"found": True, "premise": parts[0],
            "confidence": float(parts[1]),
            "justification": parts[2], ...}
```

If the LLM judges the argument self-contained, it returns the literal sentinel 'NONE | 0.0 | ...' and the UI tells the learner so. Otherwise the UI shows the premise, the confidence percentage, and a checkbox to attach the premise as an explicit supporting node when the learner submits.

Practice questions

Q. Why temperature 0.4 for premise detection but 0.7 for counter-arguments?

A. Premise detection is an analytic task — I want the LLM to be more deterministic because I'm extracting a fact about the user's argument rather than generating creative content. Counter-arguments are inherently creative and benefit from some variation, so 0.7 is appropriate there. Both are below 1.0 to avoid wildly off-topic outputs.

Q. What stops the LLM from cheating by always conceding to get high marks?

A. Nothing in the prompt directly. But the system never rewards the LLM for conceding — conceding ends the debate, and the LLM's self-reported score is not used anywhere a learner would see it as a 'win condition.' In practice the LLM concedes appropriately, but a more defensive setup would add an MDP-style reward function to the AI side too. Future work.

Part 7 — Evaluation

7.1 The 60-scenario audit

I hand-crafted 60 argumentation scenarios across four graph topologies, 15 each:

- **LIN (Linear chains)** — attack-only chains of varying depth.
- **STR (Star attacks)** — multiple attackers converging on one root.
- **BIP (Bipolar mixed)** — graphs that interleave attacks and supports.
- **LNG (Long mixed-shape)** — larger debates of 10+ nodes mixing all the above.

Every scenario has a human-expert verdict I labelled manually. The harness in `tools/run_evaluation.py` runs every scenario through the engine, thresholds the score, and compares.

Headline result

Topology	Agreement
Linear chains	12/15 (80.0%)
Star attacks	14/15 (93.3%)
Bipolar mixed	12/15 (80.0%)
Long mixed-shape	13/15 (86.7%)
Overall	51/60 (85.0%)

7.2 The ablation study

The same 60 scenarios re-run with each engine feature disabled in turn, by `tools/run_ablation.py`:

Disabled component	Verdicts changed	Mean Delta s
Support relation R_{sup}	16/60 (26.7%)	0.206
Value-tag multipliers μ	2/60 (3.3%)	0.024
Per-argument weights w	5/60 (8.3%)	0.074
Gradual (use Dung grounded)	25/60 (41.7%)	0.525

Two headline findings to memorise.

- **Bipolarity is not decorative.** Disabling the support relation flips 26.7% of verdicts.
- **Your engine genuinely differs from classical Dung.** Replacing gradual semantics with grounded extension flips 41.7% of verdicts. This is the scientific evidence that your work adds something on top of the classical baseline.

7.3 Unit tests

The PyTest suite under `tests/` contains **101 passing assertions**:

- **Engine tests** — bounds in $[0, 1]$, classical agreement on simple chains, value-tag asymmetry direction, bipolar support behaviour, convergence, diagnose helper, dict round-trip, weight clamping.
- **MDP tests** — state space is exactly 18, action space is 4, policy covers every state, losing states always pick `STEP_BACK`, high-pressure states pick `VALUE_REFRAME`, transition distributions sum to 1.

- **Scenario regression** — all 60 scenarios produce valid scores and statuses, agreement rate stays above 80%.
- **TikZ export** — empty engine returns a comment, every node appears in the output, attack and support edges use distinct styles, special LaTeX characters are escaped.

Run with `python run_tests.py` (standalone, no pytest needed) or `pytest tests/` for full parametrised output.

7.4 Discrepancy analysis

The 9 disagreements (15% of 60) fall into three patterns. Memorising these is essential because the supervisor will ask why your engine doesn't agree with your own labels.

Pattern 1: even-depth chain leniency (LIN-04, LIN-14)

On even-depth attack chains, classical Dung says root is OUT, my engine says IN. The reason: gradual semantics propagates partial scores. An attacker hit by another attacker is not fully extinguished at zero but at $1/(1+\alpha)$, leaving a residual the chain's last node cannot push to zero. So even-depth chains are more lenient than their classical counterparts.

Pattern 2: weight-asymmetry floor effect (LIN-10, BIP-08, BIP-12)

When a target is hit by an attacker several times heavier, no light counter can rescue it, even one that classical Dung would treat as undefeated. The denominator $1 + \alpha/w$ grows large with the attacker's weight, and the lightweight defender adds only a small contribution to sigma. The heavy attack creates a score floor below 0.5.

Pattern 3: bipolar supporter contribution under attack (BIP-10, LNG-09, LNG-13, STR-13)

When a supporter of the main claim is itself attacked, classical intuition says the support is neutralised. My gradual rule keeps a residual support contribution alive because the attacked supporter's score is reduced but not extinguished.

Practice answer: The three patterns are not bugs to fix. They are the empirical signature of what bipolar gradual semantics adds to classical Dung. The 15% disagreement rate is the price of having a continuous evaluator that gives learners gradient feedback rather than a binary evaluator that says only IN or OUT.

Part 8 — Self-Test Question Bank

Rehearse these aloud. The first time, try to answer without looking. The second time, refine the wording. By the third pass these should be natural.

Theory questions

Q. What is Dung's framework, in one sentence?

A. A finite set of arguments and a binary attack relation between them, with semantics for deciding which arguments survive all the attacks.

Q. What does bipolar add to Dung?

A. A parallel support relation. So arguments can both attack and reinforce each other, not just attack.

Q. What does gradual add to bipolar?

A. Continuous acceptability scores in $[0, 1]$ instead of binary IN/OUT verdicts. Computed by fixed-point iteration over the engine's update rule.

Q. What is the h-categorizer?

A. Besnard and Hunter's 2001 numerical evaluator for Dung-style frameworks. The score of an argument is 1 over 1 plus the sum of its attackers' scores. My engine reduces to it on classical inputs.

Q. Why Bench-Capon's value-based framework?

A. Real audiences care about different kinds of arguments. A fact-leaning audience finds factual attacks more compelling than emotional ones. My four value tags and the μ multipliers operationalise this audience preference.

Q. Why preference-based weights?

A. Stronger arguments should be harder to defeat. Without weights, every argument is equally important regardless of how serious or trivial its content is. Amgoud and Cayrol's framework formalises this with per-argument weights.

System questions

Q. Who decides who wins a debate?

A. The status of `Msg_1`, the Main Claim, under the Bipolar Gradual semantics. If $s(\text{Msg_1}) \geq 0.5$, Side A wins. Otherwise Side B wins. The engine recomputes the verdict after every move.

Q. What is the pedagogical guarantee of your system?

A. The LLM cannot lie about who is winning, because the LLM is never asked. The engine computes the verdict deterministically and the UI reads only the engine. So if a learner makes a knockout argument, the engine marks the AI's previous node OUT regardless of how confidently the AI's reply was phrased.

Q. How is the LLM's text constrained?

A. Through a strict pipe-separated protocol and four explicit rules (A-D) that tell the LLM to accept personal experience, score honestly, concede when there is no genuine counter, and ignore typos. The parser is forgiving enough to recover from preamble.

Q. What is the difference between the momentum bar and the verdict?

A. The momentum bar shows cumulative weight of each side's surviving arguments as a percentage. The verdict is the Main Claim's status under the engine. They can diverge: a side can have higher momentum but still lose if their Main Claim ended up OUT.

MDP questions

Q. How is the reward linked to logical validity?

A. The reward is +1, 0, or -1 depending on whether the learner's next `own_status` is WINNING, TIED, or LOSING. `own_status` comes from `derive_state()`, which reads the engine's IN/OUT verdict on the learner's main claim. So the reward is the engine's gradual-semantics verdict with a small per-hint cost added on.

Q. How do you model the learner's learning state?

A. Three buckets. `own_status` reflects the engine's verdict on their main claim. `opponent_pressure` reflects how strong the opponent's last move is. `hint_streak` reflects how many hints they have used recently. These are the minimum strategic context for picking the next hint.

Q. Why expert-informed P instead of learned P?

A. I do not have logged student trajectories yet. The codebase has a stub function `update_transition_model_from_logs` that loads transitions from logs and updates P empirically. This is explicit future work in Chapter 6.

Q. Why value iteration and not Q-learning?

A. Value iteration is exact for small discrete state spaces. With 18 states and 4 actions it converges in microseconds. Q-learning would only be needed if I could not enumerate the state-action space, which I can.

Evaluation questions

Q. What is your headline evaluation finding?

A. On 60 hand-crafted scenarios across four graph topologies, my engine agrees with my human-expert labels 85% of the time. The 15% disagreement follows three documented patterns that reflect properties of gradual semantics rather than bugs.

Q. What is your ablation finding?

A. Two important numbers. Disabling the support relation flips 26.7% of verdicts. Replacing gradual semantics with classical Dung-grounded flips 41.7%. So both bipolarity and graduality carry real signal.

Q. What did you NOT measure?

A. A formal user study, learning gain, controlled latency benchmarks, the MDP-versus-baseline hint comparison ratings. I built the comparison harness (`tools/run_hint_comparison.py`) but have not collected the human ratings yet.

Q. Why are your human-expert labels reliable?

A. They are mine, so they are biased toward what my engine would output. A stronger evaluation would use multiple independent raters and report inter-rater agreement with a kappa statistic. The rater spreadsheet for that (`tools/make_rater_template.py`) is built but not yet populated. This is in the threats to validity section.

Final answer to memorise

Closing line of the viva: 'The deepest lesson I take away from this project is that the boundary between conversational AI and formal logic is best policed by code, not by prose. Asking a language

model to be honest about whether it is losing is asking the wrong agent the wrong question. Making it unable to lie, by depriving it of the privilege of computing the verdict, turns out to be much easier and much more reliable.'

End of Study Guide. Good luck with the viva.